ASSIGNMENT-11.2

H.no-2303A51599

Batch – 29

Task 1 – Stack Using AI Guidance

Prompt :

Design and implement a Stack data structure supporting basic stack operations. A Python Stack class supporting push, pop, peek, and empty- check operations with proper documentation.

CODE:

AI-11.2.py > ...

```python
#design and implement a Stack data structure supporting basic stack operations.A Python Stack class supporting push, pop, pee
class Stack:
    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []

    def push(self, item):
        """Add an item to the top of the stack.

        Args:
            item: The item to be added to the stack.
        """
        self.stack.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack.

        Returns:
            The item at the top of the stack.

        Raises:
            IndexError: If the stack is empty.
        """
        if self.empty():
            raise IndexError("Pop from an empty stack")
        return self.stack.pop()

    def peek(self):
        """Return the item at the top of the stack without removing it.

        Returns:
            The item at the top of the stack.

        Raises:
            IndexError: If the stack is empty.
        """
        if self.empty():
            raise IndexError("Peek from an empty stack")
        return self.stack[-1]

    def empty(self):
        """Check if the stack is empty.

        Returns:
            True if the stack is empty, False otherwise.
        """
        return len(self.stack) == 0
```

```python
        if self.empty():
            raise IndexError("Peek from an empty stack")
        return self.stack[-1]

    def empty(self):
        """Check if the stack is empty.

        Returns:
            True if the stack is empty, False otherwise.
        """
        return len(self.stack) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print(stack.peek())  # Output: 3
    print(stack.pop())   # Output: 3
    print(stack.peek())  # Output: 2
    print(stack.empty()) # Output: False
    stack.pop()
    stack.pop()
    print(stack.empty()) # Output: True
```

OUTPUT:

```
3
3
2
False
True
PS D:\AI>
```

JUSTIFICATION:

A stack is an important data structure that follows the Last In First Out (LIFO) principle. It is widely used in applications such as function calls, recursion, and undo/redo operations in editors. Stacks also help in expression evaluation and syntax parsing in compilers.

Task 2 – Queue Design

Prompt :

create a Queue data structure following FIFO principles. A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

CODE:

AI-11.2.py > Queue > empty

```python
#create a Queue data structure following FIFO principles.A complete Queue implementation including enqueue, dequeue,front element acc
class Queue:
    def __init__(self):
        """Initialize an empty queue."""
        self.queue = []

    def enqueue(self, item):
        """Add an item to the end of the queue.

        Args:
            item: The item to be added to the queue.
        """
        self.queue.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue.

        Returns:
            The item at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.empty():
            raise IndexError("Dequeue from an empty queue")
        return self.queue.pop(0)

    def front(self):
        """Return the item at the front of the queue without removing it.

        Returns:
            The item at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.empty():
            raise IndexError("Front from an empty queue")
        return self.queue[0]

    def size(self):
        """Return the number of items in the queue.

        Returns:
            The size of the queue.
        """
        return len(self.queue)
```


```python
        if self.empty():
            raise IndexError("Front from an empty queue")
        return self.queue[0]

    def size(self):
        """Return the number of items in the queue.

        Returns:
            The size of the queue.
        """
        return len(self.queue)
```


```python
    def empty(self):
        """Check if the queue is empty.

        Returns:
            True if the queue is empty, False otherwise.
        """
        return len(self.queue) == 0
# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)

    print(queue.front())  # Output: 1
    print(queue.dequeue()) # Output: 1
    print(queue.front())  # Output: 2
    print(queue.size())   # Output: 2
    queue.dequeue()
    queue.dequeue()
    print(queue.empty())  # Output: True
```

OUTPUT:

```
1
1
2
2
True
PS D:\AI>
```

JUSTIFICATION:

A queue follows the First In First Out (FIFO) principle, ensuring elements are processed in the order they arrive. It is commonly used in CPU scheduling, printer task management, and network data handling. Queues help manage tasks efficiently in systems where order is important.

Task 3 – Singly Linked List Construction

Prompt :

build a singly linked list supporting insertion and traversal. Correctly functioning linked list with node creation, insertion logic, and display functionality.

CODE:

```python
#build a singly linked list supporting insertion and traversal.Correctly functioning linked list with node creation, insertion logic, and display functionality
class Node:
    def __init__(self, data):
        """Initialize a node with data and a pointer to the next node."""
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        """Initialize an empty singly linked list."""
        self.head = None

    def insert(self, data):
        """Insert a new node with the given data at the end of the list.

        Args:
            data: The data to be stored in the new node.
        """
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """Display the contents of the linked list."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()  # for a new line after displaying the list
# Example usage:
if __name__ == "__main__":
    linked_list = SinglyLinkedList()
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)

    linked_list.display()  # Output: 1 2 3
```

OUTPUT:

```
1 2 3
PS D:\AI>
```

JUSTIFICATION:

A singly linked list is a dynamic data structure that allows efficient insertion and deletion of elements. It is useful when the size of data changes frequently. Linked lists are also used in implementing stacks, queues, and memory management systems.

Task 4 – Binary Search Tree Operations

Prompt :

Implement a Binary Search Tree support focusing on insertion and traversal. BST program with correct node insertion and in-order traversal output.

CODE:

```python
#Implement a Binary Search Tree with AI support focusing on insertion and traversal.BST program with correct node insertion and in-order traversal output.
class TreeNode:
    def __init__(self, key):
        """Initialize a tree node with a key and pointers to left and right children."""
        self.left = None
        self.right = None
        self.val = key
class BinarySearchTree:
    def __init__(self):
        """Initialize an empty binary search tree."""
        self.root = None

    def insert(self, key):
        """Insert a new key into the binary search tree.

        Args:
            key: The key to be inserted into the BST.
        """
        if self.root is None:
            self.root = TreeNode(key)
        else:
            self._insert_recursively(self.root, key)

    def _insert_recursively(self, node, key):
        """Helper method to insert a key recursively."""
        if key < node.val:
            if node.left is None:
                node.left = TreeNode(key)
            else:
                self._insert_recursively(node.left, key)
        else:
            if node.right is None:
                node.right = TreeNode(key)
            else:
                self._insert_recursively(node.right, key)

    def in_order_traversal(self):
        """Perform in-order traversal of the BST and return a list of keys."""
        return self._in_order_recursively(self.root)

    def _in_order_recursively(self, node):
        """Helper method for in-order traversal recursively."""
        res = []
        if node:
            res = self._in_order_recursively(node.left)
            res.append(node.val)
```

```python
            else:
                if node.right is None:
                    node.right = TreeNode(key)
                else:
                    self._insert_recursively(node.right, key)

        def in_order_traversal(self):
            """Perform in-order traversal of the BST and return a list of keys."""
            return self._in_order_recursively(self.root)

        def _in_order_recursively(self, node):
            """Helper method for in-order traversal recursively."""
            res = []
            if node:
                res = self._in_order_recursively(node.left)
                res.append(node.val)
                res = res + self._in_order_recursively(node.right)
            return res
    # Example usage:
    if __name__ == "__main__":
        bst = BinarySearchTree()
        bst.insert(5)
        bst.insert(3)
        bst.insert(7)
        bst.insert(2)
        bst.insert(4)

        print(bst.in_order_traversal())  # Output: [2, 3, 4, 5, 7]
```

OUTPUT:

```
[2, 3, 4, 5, 7]
PS D:\AI>
```

JUSTIFICATION:

A Binary Search Tree (BST) organizes data in a sorted hierarchical structure, allowing faster searching and insertion operations. It improves efficiency compared to linear search. BSTs are widely used in database indexing, searching algorithms, and file systems.

Task 5 – Hash Table Implementation

Prompt :

Create a hash table with collision handling. Hash table supporting insert, search, and delete using chaining or open.

CODE:

```python
#Create a hash table using AI with collision handling.Hash table supporting insert, search, and delete using chaining or open.
class HashTable:
    def __init__(self, size=10):
        """Initialize a hash table with a specified size."""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table.

        Args:
            key: The key to be inserted.
            value: The value associated with the key.
        """
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                return
        self.table[index].append((key, value))  # Insert new key-value pair

    def search(self, key):
        """Search for a value by its key in the hash table.

        Args:
            key: The key to search for.

        Returns:
            The value associated with the key if found, else None.
        """
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None

    def delete(self, key):
        """Delete a key-value pair from the hash table by its key.

        Args:
            key: The key to be deleted.
```

```python
    def delete(self, key):
        """Delete a key-value pair from the hash table by its key.

        Args:
            key: The key to be deleted.

        Returns:
            True if the key was found and deleted, False otherwise.
        """
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return True
        return False
# Example usage:
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print(hash_table.search("name"))  # Output: Alice
    print(hash_table.search("age"))   # Output: 30
    print(hash_table.search("city"))  # Output: New York

    hash_table.delete("age")
    print(hash_table.search("age"))   # Output: None
```

OUTPUT:

```
Alice
30
New York
None
PS D:\AI>
```

JUSTIFICATION:

A hash table is used for fast data storage and retrieval using key-value mapping. It provides very efficient search operations with average constant time complexity O(1). Hash tables are widely used in databases, caches, dictionaries, and indexing systems.